Advanced Algorithms

Unit IV

Dr. Ayush Kumar Agrawal

UPES

Shortest Path Problem: Introduction

Core Idea

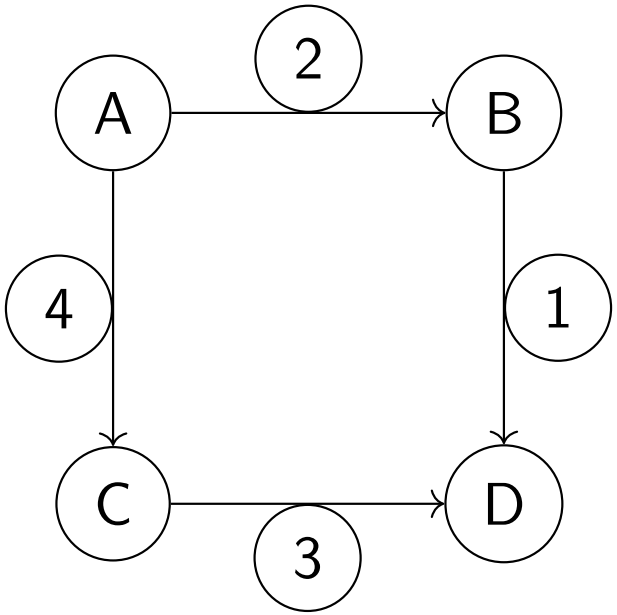
The shortest path problem aims to determine the minimum-cost path between two vertices in a graph.

- Cost may represent:
  - Distance (km)
  - Time (minutes)
  - Energy consumption
- Graph model:

$$G = (V, E, w)$$

where $w(u, v)$ is weight of edge (u, v)

Graph Representation (Example)



- Directed weighted graph
- Edge weights represent cost

Formal Problem Definition

Definition

Given graph $G = (V, E)$ and source s, find shortest path to all vertices $v \in V$.

- Path:

$$P = (v_0, v_1, ..., v_k)$$

- Path cost:

$$\sum_{i=0}^{k-1} w(v_i, v_{i+1})$$

- Objective:

$$\text{min cost of } P$$

Types of Shortest Path Problems

- **Single Source (SSSP)**
  - One source $\rightarrow$ all vertices
- **All Pairs (APSP)**
  - Every pair (i, j)
- **Single Destination**
  - All nodes $\rightarrow$ one target

Key Insight

Different problems require different algorithms.

Real-World Applications

- GPS Navigation Systems
- Internet Routing Protocols
- Robotics Path Planning
- Airline Scheduling

Example

Google Maps computes shortest or fastest routes using graph algorithms.

Example: Path Computation

Paths from A to D:

- $A \rightarrow B \rightarrow D$

$$2 + 1 = 3$$

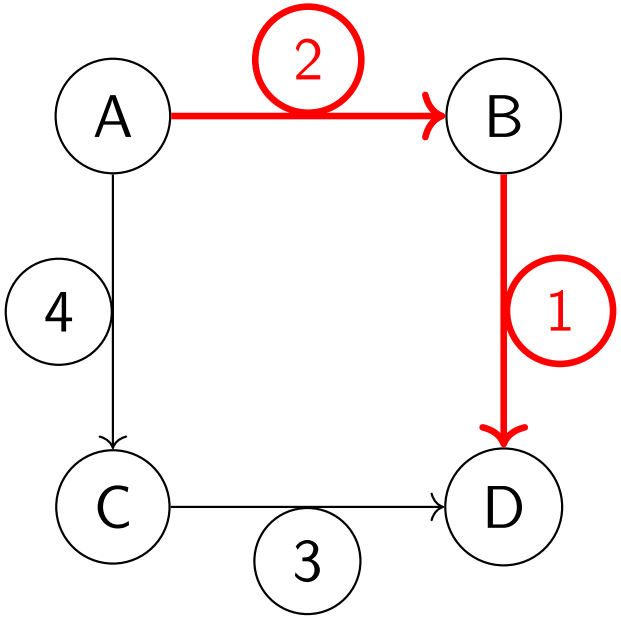
- $A \rightarrow C \rightarrow D$

$$4 + 3 = 7$$

Conclusion

Shortest path $= 3$

Visualization of Optimal Path



Observation

Highlighted path minimizes total weight.

Weighted vs Unweighted Graphs

- **Unweighted Graph**
  - All weights $= 1$
  - BFS finds shortest path
- **Weighted Graph**
  - Requires algorithms like:
    - Dijkstra
    - Bellman-Ford
    - Floyd-Warshall

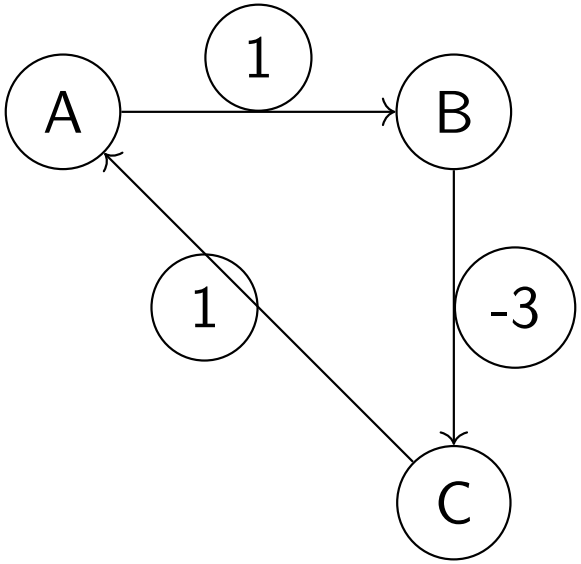
Negative Edge Weights

- Some graphs include negative weights
- Useful in:
  - Financial models
  - Gain/loss systems

Challenge

Negative weights complicate shortest path computation.

Negative Cycle Problem



- Cycle cost $= 1 - 3 + 1 = -1$

Issue

Infinite reduction in path cost $\rightarrow$ no valid solution

Shortest Path Algorithms

- **Dijkstra**
  - Greedy approach
  - No negative weights
- **Bellman-Ford**
  - Handles negative weights
  - Detects negative cycles
- **Floyd-Warshall**
  - All pairs shortest path
  - Dynamic Programming

Algorithm Comparison

Algorithm	Negative Edges	Complexity
Dijkstra	No	$O(V^2)$
Bellman-Ford	Yes	$O(VE)$
Floyd-Warshall	Yes	$O(V^3)$

Insight

Choice depends on graph size and constraints.

Why Floyd-Warshall?

- Computes shortest path between all pairs
- Works with negative weights
- Based on Dynamic Programming

Core Idea

Iteratively improve path using intermediate vertices.

Dynamic Programming Connection

- Break problem into subproblems
- Store intermediate results
- Avoid recomputation

Key Property

Optimal Substructure:

$$Shortest(i, j) = \min(Shortest(i, k) + Shortest(k, j))$$

Summary

- Shortest path is fundamental in algorithms
- Multiple problem variants exist
- Negative weights introduce challenges
- Floyd-Warshall uses DP for APSP

Next Lecture

Detailed Floyd-Warshall Algorithm with matrix formulation

Introduction to Floyd–Warshall

Definition

Floyd–Warshall is a Dynamic Programming algorithm used to compute shortest paths between all pairs of vertices.

- Works for directed weighted graphs
- Handles negative weights
- Does NOT allow negative cycles

Key Idea

Core Concept

Allow intermediate vertices step by step.

$$D^{(k)}[i][j] = \text{shortest path from i to j using vertices } \{1, ..., k\}$$

- Initially: no intermediate vertices
- Gradually include vertices

Recurrence Relation

DP Formula

$$D^{(k)}[i][j] = \min\left(D^{(k-1)}[i][j],\ D^{(k-1)}[i][k] + D^{(k-1)}[k][j]\right)$$

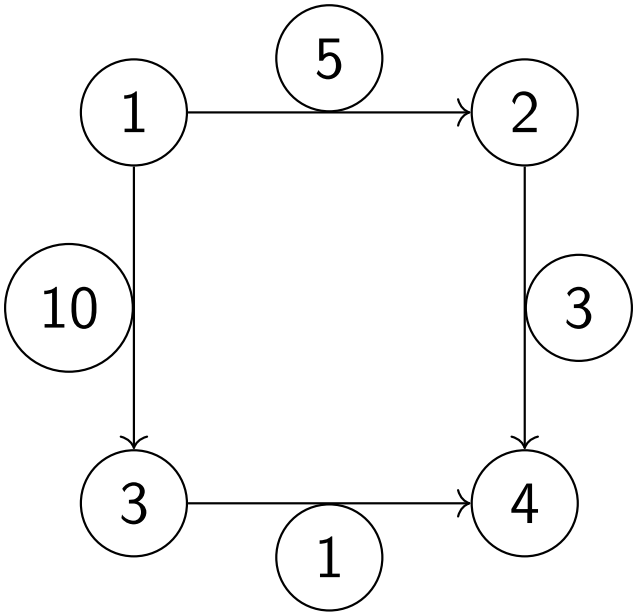
- Case 1: Do not use vertex k
- Case 2: Use vertex k as intermediate

Initial Matrix Setup

Distance Matrix

$$D[i][j] = \begin{cases} 0 & i = j \\ w(i,j) & \text{if edge exists} \\ \infty & \text{otherwise} \end{cases}$$

Example Graph



Initial Distance Matrix

$$D^{(0)} = \begin{bmatrix} 0 & 5 & 10 & \infty \\ \infty & 0 & \infty & 3 \\ \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & 0 \end{bmatrix}$$

Iteration k = 1

- Allow vertex 1 as intermediate
- Update distances:

$$D[i][j] = \min(D[i][j], D[i][1] + D[1][j])$$

Matrix After k = 1

$$D^{(1)} = \begin{bmatrix} 0 & 5 & 10 & \infty \\ \infty & 0 & \infty & 3 \\ \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & 0 \end{bmatrix}$$

Observation

No change in this step

Iteration k = 2

- Now allow vertex 2
- Check paths via vertex 2

$$D[i][j] = \min(D[i][j], D[i][2] + D[2][j])$$

Matrix After k = 2

$$D^{(2)} = \begin{bmatrix} 0 & 5 & 10 & 8 \\ \infty & 0 & \infty & 3 \\ \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & 0 \end{bmatrix}$$

Update

Path 1→2→4 gives 8

Iteration k = 3

$$D[i][j] = \min(D[i][j], D[i][3] + D[3][j])$$

- Check paths via vertex 3

Matrix After k = 3

$$D^{(3)} = \begin{bmatrix} 0 & 5 & 10 & 8 \\ \infty & 0 & \infty & 3 \\ \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & 0 \end{bmatrix}$$

Iteration k = 4

$$D[i][j] = \min(D[i][j], D[i][4] + D[4][j])$$

- Final iteration

Final Distance Matrix

$$D^{(4)} = \begin{bmatrix} 0 & 5 & 10 & 8 \\ \infty & 0 & \infty & 3 \\ \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & 0 \end{bmatrix}$$

Result

All pairs shortest paths computed

Algorithm & Complexity

Pseudocode

```
for k = 1 to n:
for i = 1 to n:
for j = 1 to n:
D[i][j] = min(D[i][j],
D[i][k] + D[k][j])
```

- Time Complexity: $O(n^3)$
- Space Complexity: $O(n^2)$

Recap of Floyd–Warshall

- Computes all-pairs shortest paths
- Based on Dynamic Programming
- Recurrence:

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$

Limitations

- Cannot handle negative cycles
- Only computes distances (not paths directly)
- Requires modification for path reconstruction

Negative Cycle Detection

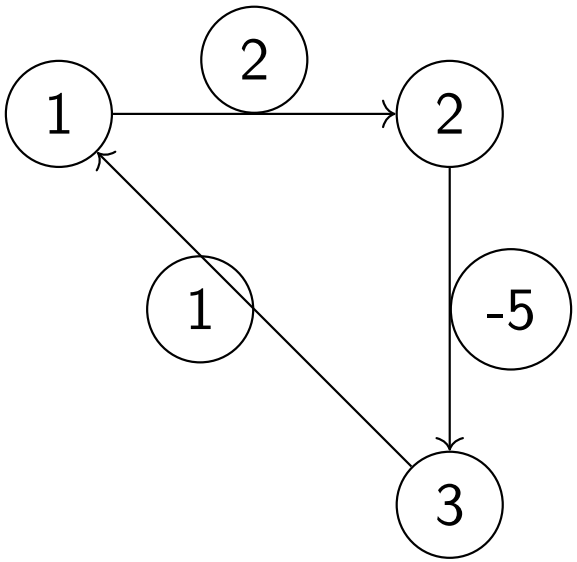
Key Observation

If any diagonal entry becomes negative:

$$D[i][i] < 0$$

then graph contains a negative cycle

Example of Negative Cycle



$$2 - 5 + 1 = -2$$

Result

Negative cycle detected

Distance Matrix Insight

- After algorithm completion:

$$D[i][i] < 0 \Rightarrow \text{Negative cycle}$$

- Important for correctness

Path Reconstruction

Problem

Floyd–Warshall gives shortest distance, but not actual path.

Solution

Maintain a **Predecessor Matrix** P

Predecessor Matrix

- $P[i][j]$ stores predecessor of j in shortest path from i
- Initialize:

$$P[i][j] = \begin{cases} i & \text{if edge exists} \\ -1 & \text{otherwise} \end{cases}$$

Updating Predecessor

When updating distance:

$$D[i][j] = D[i][k] + D[k][j]$$

Then update:

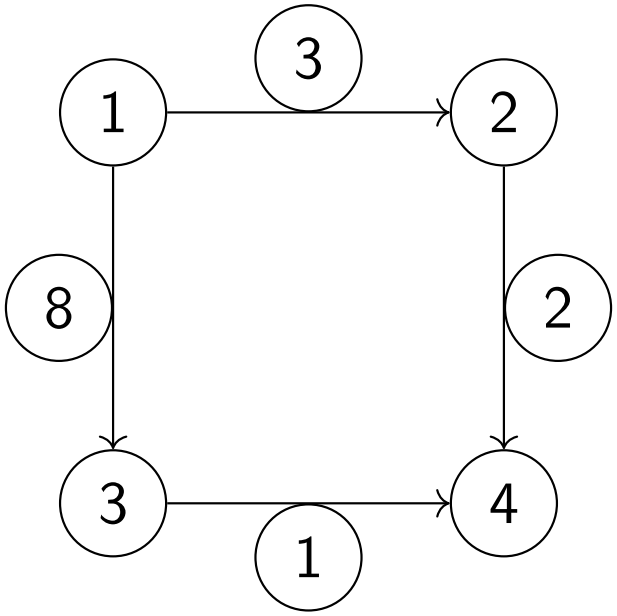
$$P[i][j] = P[k][j]$$

Path Reconstruction Process

To find path from i to j:

- Start from j
- Repeatedly follow $P[i][j]$
- Stop at i

Example Graph



Shortest Path Example

- Path $1 \rightarrow 4$:
  - $1 \rightarrow 2 \rightarrow 4 = 5$
  - $1 \rightarrow 3 \rightarrow 4 = 9$

Result

Shortest path: $1 \rightarrow 2 \rightarrow 4$

C++ Implementation

```
for(int k=0;k<n;k++)
for(int i=0;i<n;i++)
for(int j=0;j<n;j++){
 if(dist[i][k]+dist[k][j] < dist[i][j]){
  dist[i][j]=dist[i][k]+dist[k][j];
  parent[i][j]=parent[k][j];
 }
}
```

Python Implementation

```
for k in range(n):
for i in range(n):
for j in range(n):
if dist[i][k] + dist[k][j] < dist[i][j]:
dist[i][j] = dist[i][k] + dist[k][j]
parent[i][j] = parent[k][j]
```

Complexity Analysis

- Time Complexity:

$$O(n^3)$$

- Space Complexity:

$$O(n^2)$$

Note

Efficient for dense graphs

Summary

- Floyd–Warshall extended for:
  - Negative cycle detection
  - Path reconstruction
- Uses predecessor matrix
- Widely used in routing and networks

Next Lecture

Introduction to Dynamic Programming Paradigm

Introduction to Dynamic Programming

Definition

Dynamic Programming (DP) is a technique used to solve problems by breaking them into smaller overlapping subproblems and storing their results.

- Avoids recomputation
- Improves efficiency

When to Use DP?

DP is applicable when:

- Problem has **Overlapping Subproblems**
- Problem has **Optimal Substructure**

Key Idea

Solve each subproblem once and store the result

Overlapping Subproblems

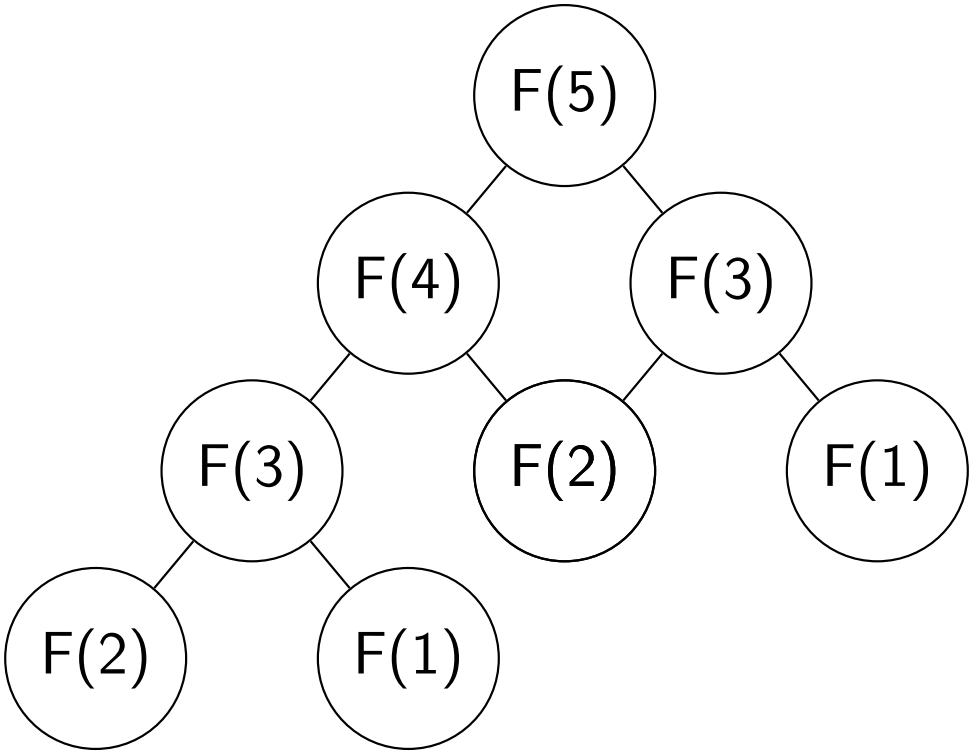
Concept

Same subproblems are solved multiple times in recursion

Example:

$$F(n) = F(n-1) + F(n-2)$$

Recursion Tree (Fibonacci)



Observation

F(3), F(2) repeated multiple times

Problem with Recursion

- Exponential time complexity:

$$O(2^n)$$

- Massive recomputation

Issue

Inefficient for large inputs

Dynamic Programming Solution

Idea

Store computed results and reuse them

- Use memory (table/array)
- Reduce time complexity

Memoization (Top-Down)

- Recursive approach
- Store results in array/map
- Compute only when needed

Formula

$$F(n) = F(n-1) + F(n-2)$$

with storage

Memoization Example

```
int fib(int n){
 if(n<=1) return n;
 if(dp[n]!=-1) return dp[n];
 return dp[n]=fib(n-1)+fib(n-2);
}
```

Tabulation (Bottom-Up)

- Iterative approach
- Build solution from base cases

Process

Start from smallest subproblems and move upward

Tabulation Example

```
int fib(int n){
 dp[0]=0; dp[1]=1;
 for(int i=2;i<=n;i++)
 dp[i]=dp[i-1]+dp[i-2];
 return dp[n];
}
```

DP Table Visualization

n	$F(n)$
0	0
1	1
2	1
3	2
4	3
5	5

Insight

Each value computed once

Optimal Substructure

Definition

Optimal solution can be built from optimal subsolutions

Example:

$$Shortest(i, j) = \min(Shortest(i, k) + Shortest(k, j))$$

DP vs Greedy vs Divide & Conquer

Technique	Key Idea	Example
DP	Store results	Knapsack
Greedy	Local choice	Dijkstra
Divide-Conquer	Split	Merge Sort

Time Complexity Improvement

- Recursive Fibonacci:

$$O(2^n)$$

- DP Fibonacci:

$$O(n)$$

Conclusion

DP drastically improves efficiency

Summary

- DP solves overlapping subproblems efficiently
- Uses memoization or tabulation
- Requires optimal substructure

Next Lecture

Classic DP Problems (Knapsack, LCS, etc.)

Classic Dynamic Programming Problems

- Fibonacci (Recap)
- 0/1 Knapsack Problem
- Longest Common Subsequence (LCS)

Goal

Understand DP formulation + table construction

Fibonacci Recap

$$F(n) = F(n-1) + F(n-2)$$

- DP reduces exponential complexity
- Stores intermediate results

0/1 Knapsack Problem

Problem Statement

Given weights and values, maximize total value without exceeding capacity.

- Each item can be taken once (0/1)

Knapsack Example

- Weights = [1, 3, 4]
- Values = [10, 40, 50]
- Capacity = 6

Goal

Maximize value

Knapsack Recurrence

$$DP[i][w] = \begin{cases} \max(DP[i-1][w],\ v_i + DP[i-1][w-w_i]) \\ \text{if } w_i \leq w \end{cases}$$

Knapsack DP Table

i/w	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10
2	0	10	10	40	50	50	50
3	0	10	10	40	50	60	90

Result

Maximum value = 90

Knapsack DP Table

0	0	0	0
0	10	10	10
0	10	40	50

- DP table showing value maximization
- Each cell represents optimal value for given weight

Knapsack Code (C++)

```
for(int i=1;i<=n;i++)
for(int w=0;w<=W;w++){
 if(wt[i]<=w)
 dp[i][w]=max(dp[i-1][w],
 val[i]+dp[i-1][w-wt[i]]);
 else
 dp[i][w]=dp[i-1][w];
}
```

Longest Common Subsequence

Definition

Find longest sequence common in two strings

Example:

$$X = ABCD, \quad Y = ACBD$$

LCS Recurrence

$$LCS[i][j] = \begin{cases} 1 + LCS[i-1][j-1] & \text{if match} \\ \max(LCS[i-1][j], LCS[i][j-1]) & \text{otherwise} \end{cases}$$

LCS Table

	A	C	B	D
A	1	1	1	1
B	1	1	2	2
C	1	2	2	2
D	1	2	2	3

Result

LCS length $= 3$

Knapsack Visualization

0	0	0	0
0	10	10	10
0	10	40	50

Observation

The table represents optimal values for each subproblem.

LCS Code (Python)

```
for i in range(1,n+1):
for j in range(1,m+1):
if X[i-1]==Y[j-1]:
dp[i][j]=1+dp[i-1][j-1]
else:
dp[i][j]=max(dp[i-1][j],
dp[i][j-1])
```

Comparison of Problems

Problem	Type	Complexity
Fibonacci	1D DP	O(n)
Knapsack	2D DP	O(nW)
LCS	2D DP	O(nm)

Summary

- DP solves optimization problems efficiently
- Knapsack: maximize value
- LCS: sequence matching

Next Lecture

Advanced DP Problems (Matrix Chain Multiplication)

Matrix Chain Multiplication (MCM)

Problem

Given a sequence of matrices, find the most efficient way to multiply them.

- Matrix multiplication is associative
- Different parenthesizations $\rightarrow$ different costs

Example

Matrices:

$$A_1(10 \times 30),\ A_2(30 \times 5),\ A_3(5 \times 60)$$

Two ways:

- $(A_1 A_2)A_3$
- $A_1(A_2 A_3)$

Cost Comparison

- $(A_1A_2)A_3$:

$$10 \times 30 \times 5 + 10 \times 5 \times 60 = 1500 + 3000 = 4500$$

- $A_1(A_2A_3)$:

$$30 \times 5 \times 60 + 10 \times 30 \times 60 = 9000 + 18000 = 27000$$

Conclusion

First order is optimal

Why Dynamic Programming?

- Overlapping subproblems
- Optimal substructure

Goal

Minimize scalar multiplications

DP Formulation

Let:

$$m[i][j] = \text{minimum cost to multiply } A_i \ldots A_j$$

$$m[i][j] = \min_{i \leq k < j} (m[i][k] + m[k+1][j] + p_{i-1} p_k p_j)$$

Base Case

$$m[i][i] = 0$$

Reason

Single matrix requires no multiplication

Computation Order

- Fill table diagonally
- Start with chain length $= 2$

Example Dimensions

$$p = [10, 30, 5, 60]$$

- $A_1 = 10 \times 30$
- $A_2 = 30 \times 5$
- $A_3 = 5 \times 60$

DP Table Initialization

	1	2	3
1	0		
2		0	
3			0

Filling Table

- Compute $m[1][2] = 1500$
- Compute $m[2][3] = 9000$

$$m[1][3] = \min(4500, 27000) = 4500$$

Final DP Table

	1	2	3
1	0	1500	4500
2		0	9000
3			0

Visualization

$$A_1 \longrightarrow A_2 \longrightarrow A_3$$

Optimal split at $k = 1$

MCM Code (C++)

```
for(int L=2;L<=n;L++){
 for(int i=1;i<=n-L+1;i++){
  int j=i+L-1;
  m[i][j]=INT_MAX;
  for(int k=i;k<j;k++){
   m[i][j]=min(m[i][j],
   m[i][k]+m[k+1][j]+p[i-1]*p[k]*p[j]);
  }
 }
}
```

Connection to Floyd–Warshall

- Both use DP tables
- Both use intermediate splitting

$$Shortest(i, j) = \min(Shortest(i, k) + Shortest(k, j))$$

Insight

General DP paradigm: try all possible splits

Summary

- MCM optimizes matrix multiplication order
- Uses DP with $O(n^3)$ time
- Key idea: optimal split point

Next Lecture

Modular Arithmetic and Chinese Remainder Theorem

Introduction to Modular Arithmetic

Definition

For integers a and n:

$$a \equiv b \pmod{n}$$

means $n \mid (a - b)$

- Represents remainder after division

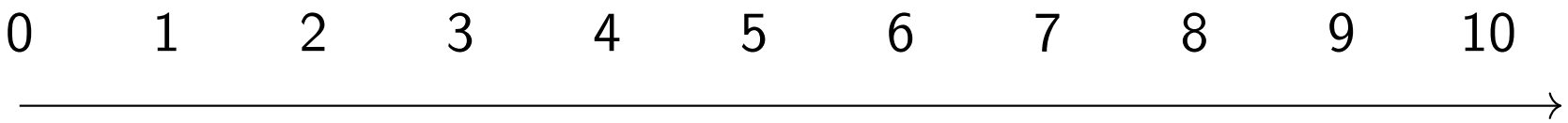
Examples

- $17 \equiv 2 \pmod 5$
- $23 \equiv 3 \pmod{10}$

Interpretation

Remainder after division

Modular Number Line



- Modulo wraps values cyclically

Properties of Modular Arithmetic

- Addition:

$$(a+b) \mod n = (a \mod n + b \mod n) \mod n$$

- Multiplication:

$$(ab) \mod n = (a \mod n \cdot b \mod n) \mod n$$

Modular Inverse

Definition

a^{-1} such that:

$$a \cdot a^{-1} \equiv 1 \pmod{n}$$

- Exists if $\gcd(a, n) = 1$

Chinese Remainder Theorem

Statement

If $n_1, n_2, ..., n_k$ are pairwise coprime, then system:

$$x \equiv a_i \pmod{n_i}$$

has unique solution modulo $N = n_1 n_2 ... n_k$

CRT Example

Solve:

$$x \equiv 2 \pmod 3$$

$$x \equiv 3 \pmod 5$$

Step 1: Compute Product

$$N = 3 \times 5 = 15$$

$$N_1 = 5, \quad N_2 = 3$$

Step 2: Find Inverses

$$5^{-1} \equiv 2 \pmod 3$$

$$3^{-1} \equiv 2 \pmod 5$$

Step 3: Construct Solution

$$x = 2 \cdot 5 \cdot 2 + 3 \cdot 3 \cdot 2$$

$$x = 20 + 18 = 38$$

$$x \equiv 8 \pmod{15}$$

Verification

- $8 \mod 3 = 2$
- $8 \mod 5 = 3$

Correct Solution

$x = 8$

Why CRT is Useful

- Large number computations
- Cryptography (RSA)
- Parallel computations

Algorithm (General Form)

```
x = 0
for each equation:
compute Ni = N/ni
find inverse Mi
x += ai * Ni * Mi
return x % N
```

Complexity

- Depends on inverse computation
- Typically:

$$O(k \log n)$$

Summary

- Modular arithmetic simplifies computation
- CRT solves simultaneous congruences
- Unique solution exists for coprime moduli

Next Lecture

Conversion between base and modular representation

Motivation

Key Idea

Efficient computation can be achieved by changing number representation.

- Base representation (decimal, binary)
- Modular representation (CRT)

Base Representation

Definition

A number N in base b:

$$N = a_0 + a_1 b + a_2 b^2 + \cdots + a_k b^k$$

Example:

$$345 = 3 \times 10^2 + 4 \times 10^1 + 5$$

Visualization

$$345$$

$$= 3 \times 10^2 + 4 \times 10 + 5$$

Modular Representation

Idea

Represent number using residues:

$$(N \mod n_1, N \mod n_2, ..., N \mod n_k)$$

Example:

$$17 \rightarrow (2, 2) \text{ mod } (3, 5)$$

Why Modular Representation?

- Parallel computation
- Faster arithmetic operations
- Avoid large numbers

Conversion: Base $\rightarrow$ Modular

- Compute:

$$N \mod n_i$$

for each modulus

Example

$$N = 17,\ (mod\ 3, 5) = (2, 2)$$

Conversion: Modular $\rightarrow$ Base

Using CRT

Recover N from residues:

$$N \equiv a_i \pmod{n_i}$$

Polynomial Representation

Definition

A polynomial:

$$P(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

- Similar to base representation
- Variable x replaces base

Coefficient vs Point Representation

- Coefficient form:

$$(a_0, a_1, ..., a_n)$$

- Value form:

$$(P(x_1), P(x_2), ..., P(x_n))$$

Key Idea

FFT converts between these forms

Example Polynomial

$$P(x) = 2 + 3x + x^2$$

Evaluate at:

$$x = 1, 2, 3$$

$$P(1) = 6,\ P(2) = 12,\ P(3) = 20$$

Interpolation Problem

Goal

Given points (x_i, y_i), find polynomial $P(x)$

Lagrange Interpolation

$$P(x) = \sum_{i=0}^{n} y_i L_i(x)$$

$$L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

Example

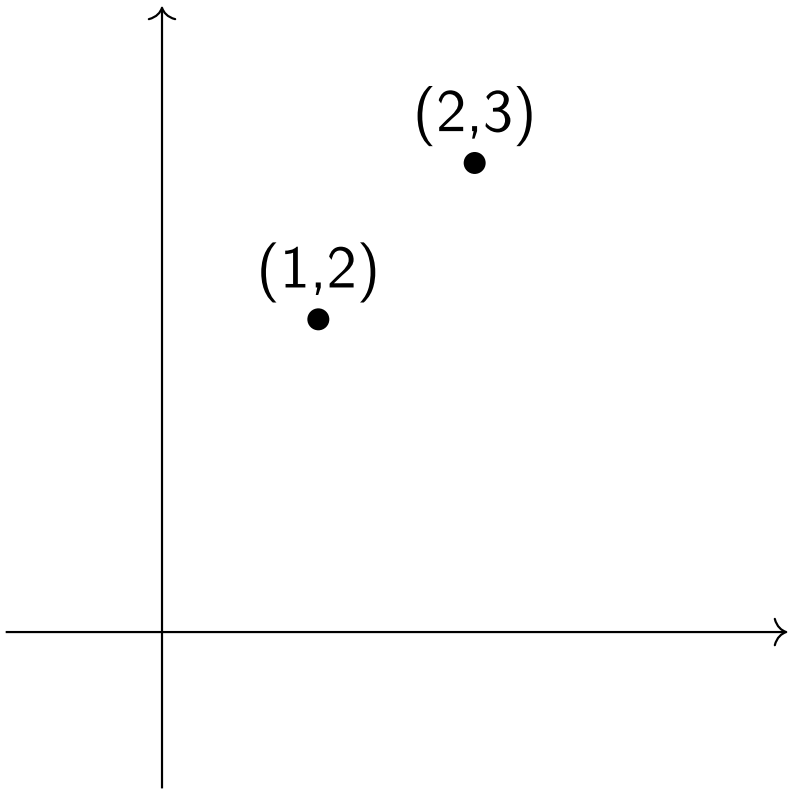
Points:

$$(1, 2), (2, 3)$$

$$P(x) = 2\frac{x-2}{1-2} + 3\frac{x-1}{2-1}$$

$$P(x) = -2(x-2) + 3(x-1)$$

Visualization



Summary

- Base vs modular representation
- Polynomial representation forms
- Interpolation reconstructs polynomial

Next Lecture

Discrete Fourier Transform (DFT)

Introduction to DFT

Definition

Discrete Fourier Transform (DFT) converts a sequence into its frequency representation.

- Used for fast polynomial multiplication
- Basis for FFT

Motivation

- Polynomial multiplication is expensive:

$$O(n^2)$$

- DFT reduces it to:

$$O(n \log n)$$

Complex Numbers

Definition

$$z = a + ib$$

- Magnitude: $|z| = \sqrt{a^2 + b^2}$
- Euler's formula:

$$e^{i\theta} = \cos\theta + i\sin\theta$$

Roots of Unity

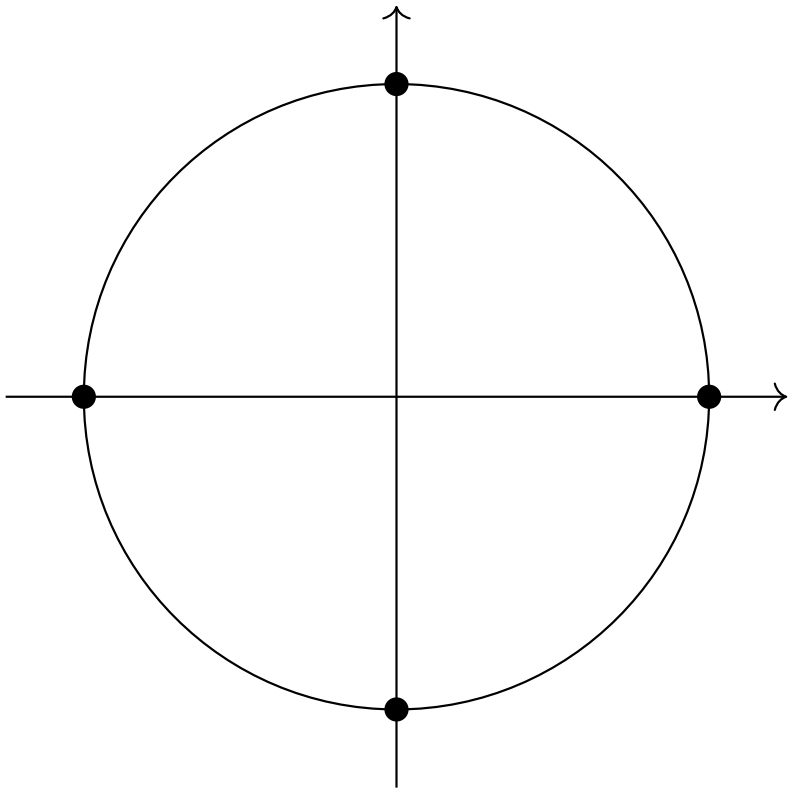
Definition

$$\omega_n = e^{2\pi i/n}$$

$$\omega_n^k = e^{2\pi ik/n}$$

- $\omega_n^n = 1$

Unit Circle



- 4th roots of unity

DFT Definition

Formula

$$X[k] = \sum_{j=0}^{n-1} x[j] \cdot \omega_n^{jk}$$

- Input: sequence $x[j]$
- Output: transformed sequence $X[k]$

Inverse DFT

$$x[j] = \frac{1}{n} \sum_{k=0}^{n-1} X[k] \cdot \omega_n^{-jk}$$

Insight

DFT is reversible

Polynomial Interpretation

- Given polynomial:

$$P(x) = a_0 + a_1 x + \cdots + a_{n-1} x^{n-1}$$

- DFT evaluates:

$$P(\omega_n^k)$$

Example

$$P(x) = 1 + 2x + 3x^2$$

Evaluate at roots of unity:

$$x = 1, -1, i, -i$$

Computation Insight

- Direct DFT takes:

$$O(n^2)$$

- Repeated computations exist

Problem

Inefficient for large inputs

Matrix Form

$$\begin{bmatrix} X[0] \\ X[1] \\ \vdots \\ X[n-1] \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & \cdots \\ 1 & \omega & \omega^2 & \cdots \\ \vdots & & & \end{bmatrix} \begin{bmatrix} x[0] \\ x[1] \\ \vdots \end{bmatrix}$$

Key Properties

- Periodicity:

$$\omega_n^{k+n} = \omega_n^k$$

- Symmetry
- Conjugate pairs

Application: Polynomial Multiplication

Steps:

- Convert to value form using DFT
- Multiply point-wise
- Apply inverse DFT

DFT in Modular Arithmetic

- Use finite fields instead of complex numbers
- Important for computer implementation

Summary

- DFT converts polynomial to value representation
- Uses roots of unity
- Basis for FFT

Next Lecture

Fast Fourier Transform (FFT)

Introduction to FFT

Definition

Fast Fourier Transform (FFT) is an efficient algorithm to compute the DFT in $O(n \log n)$ time.

- Improves over DFT ($O(n^2)$)
- Based on divide and conquer

Motivation

- DFT has redundant computations
- FFT eliminates redundancy

Goal

Compute polynomial evaluation faster

Key Idea

Split polynomial into:

- Even indexed terms
- Odd indexed terms

$$P(x) = P_e(x^2) + xP_o(x^2)$$

Even-Odd Decomposition

$$P(x) = (a_0 + a_2x^2 + a_4x^4) + x(a_1 + a_3x^2)$$

- Reduces problem size by half

Recursive Formula

$$P(\omega^k) = P_e(\omega^{2k}) + \omega^k P_o(\omega^{2k})$$

$$P(\omega^{k+n/2}) = P_e(\omega^{2k}) - \omega^k P_o(\omega^{2k})$$

Butterfly Operation

Core Computation

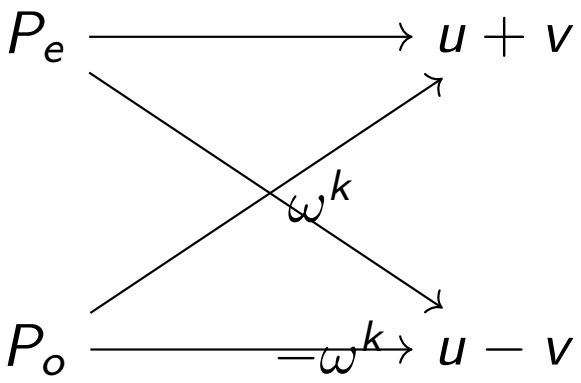
Combine results:

$$u = P_e, \quad v = \omega^k P_o$$

$$X[k] = u + v$$

$$X[k + n/2] = u - v$$

Butterfly Diagram

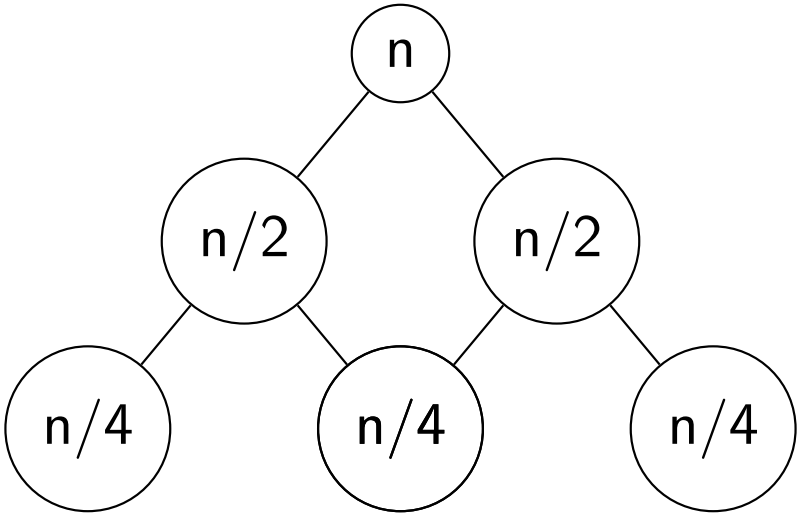


Recursive Structure

- Divide problem into two halves
- Recursively compute FFT

$$T(n) = 2T(n/2) + O(n)$$

Recursion Tree



Complexity Analysis

$$T(n) = 2T(n/2) + n$$

Using Master Theorem:

$$T(n) = O(n \log n)$$

Improvement

From $O(n^2)$ to $O(n \log n)$

FFT Algorithm (Pseudo-code)

```
FFT(a):
if n == 1 return a
split into even and odd
compute FFT recursively
combine using roots of unity
```

Polynomial Multiplication

Steps:

- Apply FFT to both polynomials
- Multiply point-wise
- Apply inverse FFT

Advantages of FFT

- Fast computation
- Used in signal processing
- Large integer multiplication

Applications

- Image processing
- Audio compression
- Cryptography

Summary

- FFT reduces DFT complexity
- Uses divide and conquer
- Core idea: even-odd decomposition

Next Lecture

Schonhage-Strassen Integer Multiplication

Introduction

Problem

Multiply very large integers efficiently

- Classical multiplication: $O(n^2)$
- FFT-based: $O(n \log n)$

Motivation

- Cryptography (RSA)
- Scientific computing
- Big integer libraries

Goal

Faster multiplication for huge numbers

Idea of FFT Multiplication

- Represent integers as polynomials
- Multiply using FFT
- Convert back to integer

Integer as Polynomial

Example:

$$1234 = 1x^3 + 2x^2 + 3x + 4$$

- Base $= 10$
- Coefficients $=$ digits

Problem with Complex FFT

- Floating-point errors
- Precision issues

Solution

Use modular arithmetic instead

Modular FFT

Concept

Perform FFT in modular ring:

$$\mathbb{Z}/n\mathbb{Z}$$

- Avoid floating-point errors
- Use integer arithmetic

Roots of Unity in Modulo

- Need primitive root ω
- Must satisfy:

$$\omega^n \equiv 1 \mod m$$

Schonhage–Strassen Idea

- Use FFT over modulo ring
- Carefully choose modulus
- Perform cyclic convolution

Algorithm Steps

1. Split integers into chunks
2. Represent as polynomials
3. Apply modular FFT
4. Multiply point-wise
5. Apply inverse FFT
6. Reconstruct integer

Visualization

Integer A

↓

Polynomial

↓

FFT

↓

Multiply

↓

Inverse FFT

Time Complexity

$$O(n \log n \log \log n)$$

Comparison

- Classical: $O(n^2)$
- Karatsuba: $O(n^{1.58})$
- FFT: $O(n \log n)$

Advantages

- Extremely fast for large inputs
- Used in high-performance libraries

Limitations

- Complex implementation
- Large constant factors

Applications

- Cryptography
- Scientific simulations
- Signal processing

Summary

- Uses FFT in modular arithmetic
- Handles very large integers efficiently
- Key algorithm in advanced computing

Unit Complete

You have covered one of the most advanced topics in algorithms

Unit VI Overview

- Shortest Path (Floyd–Warshall)
- Dynamic Programming
- Modular Arithmetic & CRT
- Polynomial Representation
- DFT and FFT
- Schonhage–Strassen Algorithm

Big Picture

Graphs

$\downarrow$

Dynamic Programming

$\downarrow$

Modular Arithmetic

$\downarrow$

DFT/FFT

$\downarrow$

Fast Multiplication

Concept Integration

Key Flow

- DP $\rightarrow$ Optimization
- CRT $\rightarrow$ Efficient representation
- FFT $\rightarrow$ Fast computation
- Multiplication $\rightarrow$ Real-world application

Floyd–Warshall Recap

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$

- All-pairs shortest path
- Uses DP paradigm

Dynamic Programming Recap

- Overlapping subproblems
- Optimal substructure
- Memoization vs Tabulation

CRT Recap

$$x \equiv a_i \pmod{n_i}$$

- Unique solution exists
- Used in modular computations

Polynomial Representation

- Coefficient form
- Value form

$$P(x) = a_0 + a_1 x + \cdots + a_n x^n$$

DFT Recap

$$X[k] = \sum x[j]\omega^{jk}$$

- Converts polynomial to value form

FFT Recap

- Divide and conquer
- Even-odd split

$$O(n \log n)$$

Schonhage–Strassen Recap

- Uses modular FFT
- Fast integer multiplication

$$O(n \log n \log \log n)$$

End-to-End Pipeline

$$\text{Integer} \rightarrow \text{Polynomial} \longrightarrow \text{FFT} \longrightarrow \text{Multiply} \rightarrow \text{Inverse FFT}$$

Comparison Table

Technique	Time Complexity	Use
DP	Varies	Optimization
CRT	$O(k \log n)$	Modular systems
FFT	$O(n \log n)$	Polynomial mult.
SS Algo	$O(n \log n \log \log n)$	Large integers

Applications

- GPS and routing systems
- Cryptography
- Signal processing
- Robotics and AI

Exam Strategy

- Focus on DP formulations
- Practice CRT problems
- Understand FFT flow clearly
- Revise complexity analysis

Final Summary

- Unit IV covers advanced algorithms
- Strong link between math and computation
- Key for research and innovation

Congratulations

You have completed one of the most advanced units in AA